

Design Report — End-to-End Analytics Engineering Pipeline

Sharon Nasimuyu

Table of Contents

Scope: Near-real-time analytics pipeline from a public REST API through an OLTP database, CDC streaming, ClickHouse, and layered dbt transformations to analytics- and ML-ready datasets.

Data source: World Bank Indicators REST API — <https://api.worldbank.org/v2/> (public, no authentication). Primary endpoints: `/v2/country` (country dimension), `/v2/indicator` (indicator dimension), and `/v2/country/{code}/indicator/{code}?format=json&per_page=...` (annual observation values per country/indicator). The design pattern is source-agnostic; the World Bank dataset — indicator values per country per year — is used to make the schema concrete because its time-series shape best demonstrates ClickHouse partitioning, ordering, and materialized-view choices.

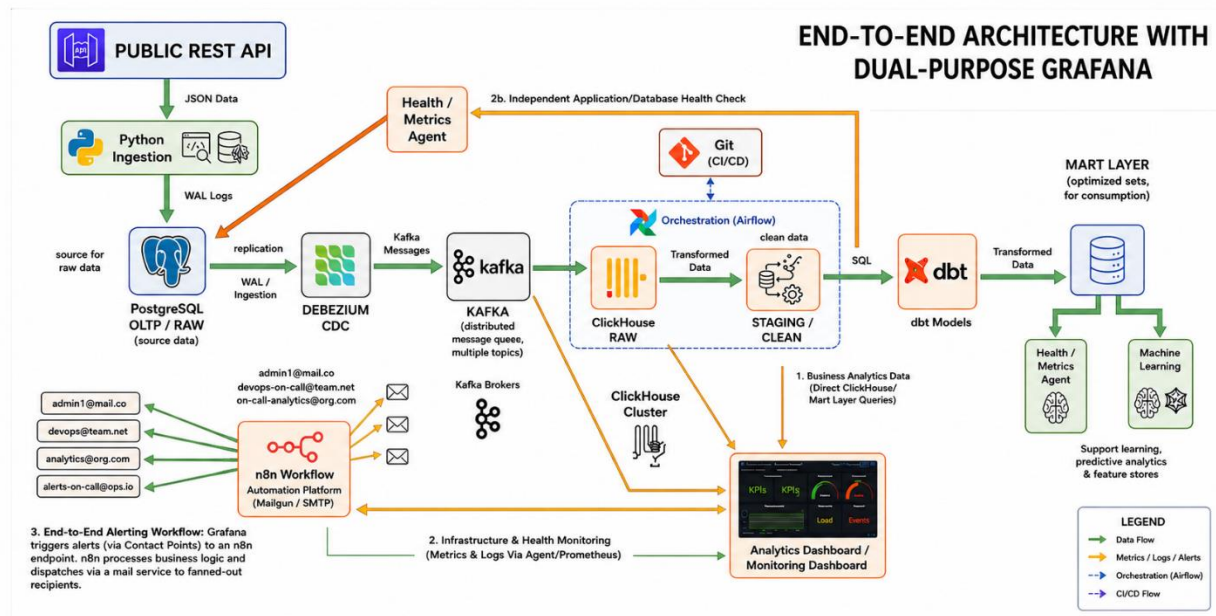
1. Overview

The pipeline ingests operational data from the World Bank REST API into PostgreSQL (OLTP), captures every row-level change with Debezium, streams those changes through Kafka into ClickHouse (OLAP), and refines the raw change stream into **staging** (clean, current-state) and **mart** (analytics- and ML-ready) layers with dbt. Airflow orchestrates the batch stages; CDC runs continuously. The full stack is containerized with Docker Compose and validated on every change by a GitHub Actions CI/CD pipeline. Prometheus, Grafana, and n8n provide observability and alerting.

Design priorities, in order: **correctness** (idempotent, deduplicated, CDC-aware loads), **near-real-time freshness**, **query performance** on ClickHouse, and **operability** (everything monitored, one-command startup, tested on every commit).

2. Architecture Diagram

End-to-End Architecture: Data Ingestion and Transformation



End-to-end architecture

Component responsibilities

Layer	Technology	Responsibility
Ingestion	Python (requests/httpx)	Pull REST API, paginate, validate, idempotent upsert into OLTP
OLTP	PostgreSQL	Source of truth; logical replication enabled (wal_level=logical)
CDC	Debezium (Kafka Connect)	Capture INSERT/UPDATE/DELETE from WAL
Streaming	Apache Kafka	Durable, decoupled transport; one topic per source table
OLAP	ClickHouse	Raw CDC landing + analytical storage
Transformation	dbt	Raw → staging → mart, with tests
Orchestration	Airflow	Schedule/monitor ingestion + dbt; retries, backfills
Containerization	Docker Compose	One-command startup of the whole stack
CI/CD	GitHub Actions	Lint, unit tests, dbt build/test on every push/PR
Observability	Prometheus + Grafana + n8n	Metrics, dashboards, alert fan-out

Data quality

dbt tests + Great
Expectations

Validation gates at each stage

3. Data Flow Explanation

1. REST API → PostgreSQL. A Python ingestion service calls the World Bank API — first the reference endpoints (`/v2/country`, `/v2/indicator`) to populate dimensions, then `/v2/country/{code}/indicator/{code}` for observation values. It handles the API's page-based pagination (the response envelope carries `page`, `pages`, `per_page`, `total`), performs basic response/schema validation, and upserts records into PostgreSQL. Ingestion is **idempotent** (upsert on the natural key `country_code` + `indicator_code` + `obs_year`), so reruns never create duplicates. PostgreSQL is the OLTP source of truth, with primary keys and timestamps supporting incremental ingestion.

2. PostgreSQL → Debezium → Kafka. With `wal_level=logical`, Debezium reads the write-ahead log and emits INSERT/UPDATE/DELETE events rather than polling. Events are published to Kafka topics (one per table). Kafka provides a durable, replayable, decoupled buffer between OLTP and downstream consumers, enabling near-real-time propagation without heavy database polling. This matters for World Bank data because indicator values are **revised historically** — an update to a prior year's figure flows through as a CDC UPDATE rather than requiring a full reload.

3. Kafka → ClickHouse RAW. ClickHouse consumes the CDC topics (via the ClickHouse **Kafka table engine** feeding a materialized view, or the Kafka Connect ClickHouse sink). The raw layer stores the full CDC payload — the changed row **plus** operation metadata (`op`, `ts_ms`, `source LSN`) — giving an auditable, replayable record of every source change.

4. RAW → Staging. dbt flattens the nested CDC envelope, standardizes names and types, normalizes timestamps, applies CDC semantics (collapse to the latest version per key, honor deletes), removes duplicates, and runs basic data-quality checks. Staging is a clean representation of **current source state**, decoupling ingestion mechanics from analytical modeling.

5. Staging → Mart. dbt applies business logic — derived metrics (year-over-year growth, CAGR), regional/income-group aggregations, dimensional joins to country and indicator attributes, feature engineering, and standardized definitions — producing analytics-ready mart tables optimized for ClickHouse. Marts feed both BI/analytics and ML/feature consumption.

6. Orchestration. Airflow coordinates ingestion → staging → mart with scheduling, dependency management, retries, failure handling, and execution visibility. CDC runs continuously; batch ingestion and dbt runs are scheduled (e.g., daily, since World Bank data updates infrequently) and gated by data-quality tests.

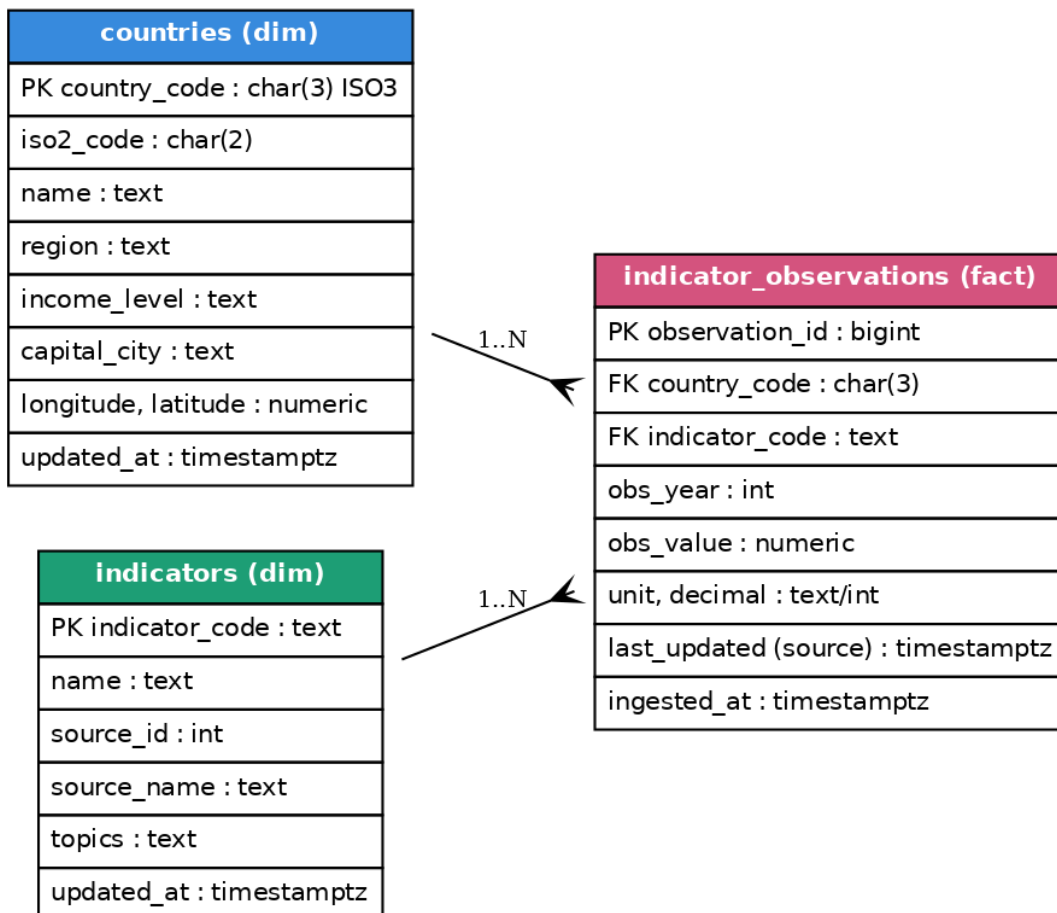
7. Data quality (throughout). Validation is applied at every hop — API response validation, schema/type checks, PK uniqueness, null checks, record-count and reconciliation checks, referential integrity (every observation references a known country

and indicator), freshness, and CDC-lag monitoring — so invalid data cannot silently propagate downstream.

4. Data Model / Schema Documentation

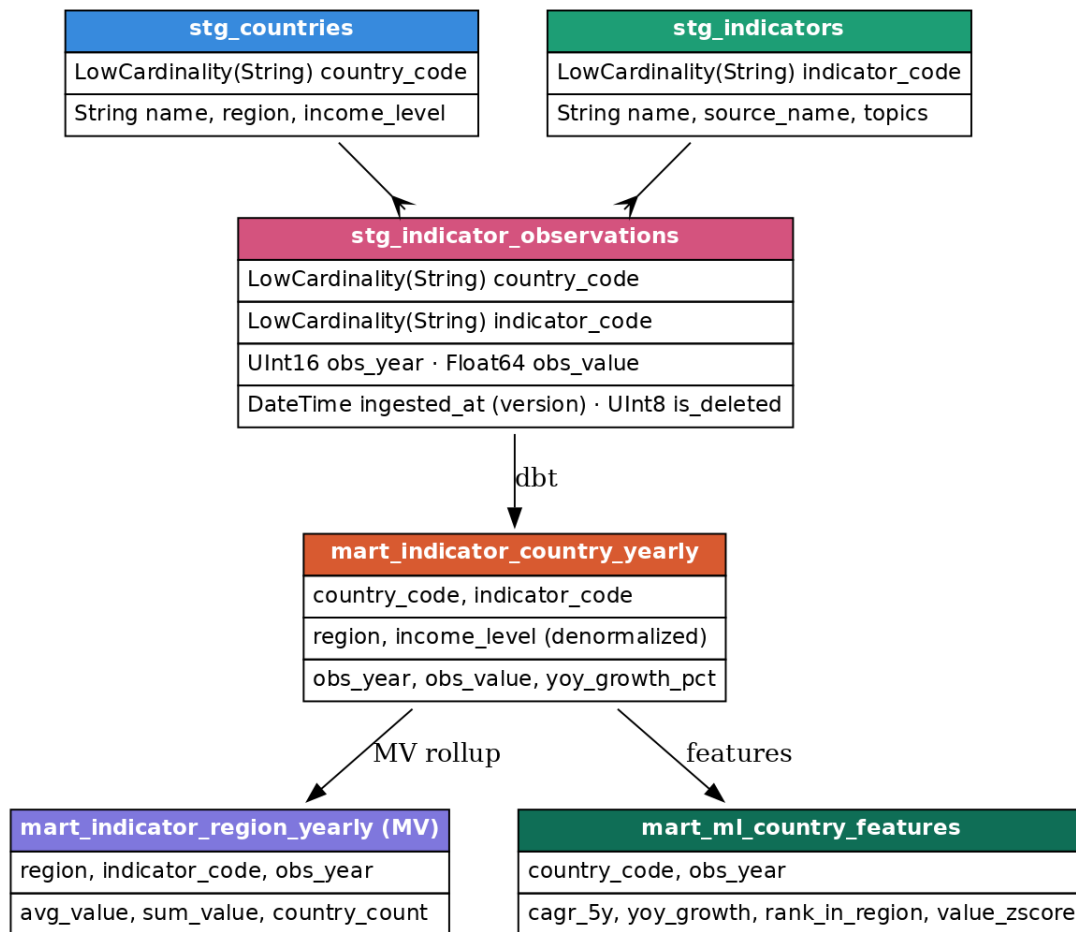
4.1 Source → OLTP (PostgreSQL)

The API exposes two reference collections (countries, indicators) and one observation series (indicator value per country per year). In OLTP this is modeled as two dimensions and one fact.



OLTP entity-relationship diagram

4.2 Staging & Mart (ClickHouse) ERD



Staging and mart entity-relationship diagram

4.3 ClickHouse-specific design choices (rationale)

RAW landing table — capture everything, deduplicate later.

```
CREATE TABLE raw.indicator_observations_cdc
(
    country_code    String,
    indicator_code  String,
    payload         String,           -- full JSON envelope
    op              LowCardinality(String), -- c/u/d/r
    source_ts_ms    UInt64,
    ingested_at     DateTime DEFAULT now()
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(ingested_at)
ORDER BY (indicator_code, country_code, source_ts_ms);
```

Engine: plain MergeTree — the raw layer is an immutable, append-only audit log; we never mutate it, only read and transform forward.

STAGING table — collapse CDC to current state.

```
CREATE TABLE staging.stg_indicator_observations
(
    country_code    LowCardinality(String),
    indicator_code  LowCardinality(String),
    obs_year        UInt16 CODEC(DoubleDelta, ZSTD(1)),
    obs_value       Float64 CODEC(ZSTD(1)),
    ingested_at     DateTime,                -- version
    is_deleted      UInt8
)
ENGINE = ReplacingMergeTree(ingested_at)
PARTITION BY intDiv(obs_year, 10) * 10      -- decade
ORDER BY (indicator_code, country_code, obs_year);
```

- **Table engine — ReplacingMergeTree(ingested_at):** the World Bank revises historical figures, so CDC produces multiple versions of the same (indicator, country, year). ReplacingMergeTree collapses rows sharing the ORDER BY key, keeping the row with the highest version column (ingested_at). This yields an efficient “latest known value” without manual dedup logic; reads use FINAL or a dedup pattern to guarantee the latest version. The is_deleted tombstone lets deletes be filtered downstream.
- **Partitioning key — intDiv(obs_year, 10) * 10 (decade):** the data is annual and spans ~1960 to present (65 years). Partitioning by decade keeps the partition count in single digits — avoiding the ClickHouse anti-pattern of many tiny partitions — while still enabling partition **pruning** on year-range queries and cheap retention via DROP PARTITION. (Monthly partitioning, appropriate for high-frequency sources, would be wasteful here.)
- **Ordering / sorting key — (indicator_code, country_code, obs_year):** matches the dominant query patterns (“this indicator across countries” and “this country/indicator over time”). ClickHouse’s sparse primary index skips granules efficiently, and physically co-locating each indicator’s series maximizes compression and range-scan speed. For a ReplacingMergeTree, the ORDER BY is the dedup key, so this also defines row identity: one indicator, one country, one year.
- **Column codecs:** DoubleDelta on the monotonic obs_year, wrapped in ZSTD, plus ZSTD on obs_value, shrink the time-series columns significantly. LowCardinality(String) for country_code/indicator_code cuts storage and speeds grouping/filtering (there are only ~200 countries and a bounded indicator set).

MART — pre-aggregate at insert time with a materialized view.

```
CREATE TABLE mart.indicator_region_yearly
(
    region          LowCardinality(String),
    indicator_code  LowCardinality(String),
```

```

    obs_year      UInt16,
    avg_value      AggregateFunction(avg, Float64),
    sum_value      AggregateFunction(sum, Float64),
    country_count  AggregateFunction(uniq, String)
)
ENGINE = AggregatingMergeTree
PARTITION BY intDiv(obs_year, 10) * 10
ORDER BY (region, indicator_code, obs_year);

CREATE MATERIALIZED VIEW mart.indicator_region_yearly_mv
TO mart.indicator_region_yearly AS
SELECT
    c.region                AS region,
    o.indicator_code        AS indicator_code,
    o.obs_year              AS obs_year,
    avgState(o.obs_value)   AS avg_value,
    sumState(o.obs_value)   AS sum_value,
    uniqState(o.country_code) AS country_count
FROM staging.stg_indicator_observations AS o
INNER JOIN staging.stg_countries AS c USING (country_code)
GROUP BY region, indicator_code, obs_year;

```

- **Materialized views:** regional/income-group rollups are computed **incrementally at insert time** by an AggregatingMergeTree-backed MV, so dashboards read tiny pre-aggregated tables instead of scanning the full observation history. Queries finalize aggregate states with the -Merge combinators (or FINAL).
- **Why not compute on read:** aggregating the full fact per dashboard load is wasteful at scale; MVs shift the cost to write time (once) and keep reads $O(\text{regions} \times \text{years})$ rather than $O(\text{observations})$.
- **Projections (optional):** for an alternate access path (e.g. “top countries by an indicator in year Y”), a projection provides a second physical ordering without a second table.

Analytics-ready fact. `mart.indicator_country_yearly` (a dbt model, MergeTree ordered by `(indicator_code, country_code, obs_year)`) denormalizes country attributes (region, income level) onto the fact and adds derived metrics such as year-over-year growth — the primary table for BI queries.

ML-ready table. `mart.ml_country_features` (a dbt incremental model, MergeTree ordered by `(country_code, obs_year)`) materializes point-in-time features (CAGR, YoY growth, rank within region, value z-score) with no leakage, ready for a feature store or direct training pulls.

5. Observability Design

Observability covers **data health** (is the data correct, fresh, complete?) and **platform health** (are the services up and performant?). Metrics flow Prometheus → Grafana

(dashboards + alert rules) → n8n (routing/business logic) → email fan-out to on-call inboxes.

What is monitored

Category	Signals	Source / exporter
Pipeline health	Airflow DAG/task success/failure, task duration, retries, SLA misses	Airflow StatsD → statsd_exporter; dbt run results
Data freshness	now() - max(ingested_at) per table; dbt source freshness	Custom ClickHouse exporter / Pushgateway
CDC lag	Debezium MilliSecondsBehindSource, Kafka Connect task state; Kafka consumer-group lag; Postgres replication-slot lag	Kafka Connect JMX exporter; kafka_exporter / Burrow; postgres_exporter
Throughput / volume	Rows ingested, Kafka messages/sec per topic, rows written to ClickHouse	Ingestion app metrics; Kafka JMX; ClickHouse system.*
Resource usage	CPU, memory, disk, IO per container/host; ClickHouse merges, parts count, memory	node_exporter, cAdvisor, ClickHouse exporter
Data quality	dbt test pass/fail, Great Expectations checkpoint results, reconciliation counts	dbt artifacts, GE → Pushgateway
Service up/down	Liveness of Postgres, Kafka, Debezium, ClickHouse, Airflow, dbt	Prometheus up, blackbox probes

Alerting examples (Grafana → n8n → email)

- **CDC lag** MilliSecondsBehindSource > 60s for 5m → warn; > 5m → page on-call.
- **Freshness** no new rows in stg_indicator_observations after a scheduled ingestion window → alert analytics + on-call.
- **Pipeline** any Airflow task failed or dbt test error → alert with DAG/model context.
- **Resource** ClickHouse parts_to_throw_insert approaching limit, or disk > 85% → alert platform.

n8n applies routing/de-duplication/formatting and dispatches via Mailgun/SMTP so the right team gets a clean, contextual email rather than raw firing alerts.

Why these tools

- **Prometheus** — pull-based scraping, a mature exporter ecosystem for every component here (Postgres, Kafka, ClickHouse, node, cAdvisor, Airflow via StatsD), PromQL for flexible SLO queries, and cheap to run in Compose.
- **Grafana** — first-class Prometheus support, reusable dashboards, and built-in alerting with contact points → natural webhook hand-off to n8n.

- **n8n** — decouples *alert generation* from *notification logic*: business rules, throttling, and multi-channel fan-out live in one automation layer instead of being hard-coded per tool.
- **dbt tests + Great Expectations** — dbt tests gate the transformation graph (not-null, unique, relationships, accepted-values, freshness); GE adds richer distribution/expectation checks. Failures become metrics, closing the loop between data quality and alerting.

6. Scaling & Extension

The design scales along four axes without re-architecture.

1. Ingestion & sources. Parallelize API pulls with async workers and rate-limit backoff; add new indicators or entirely new sources by dropping in new ingestion tasks and Kafka topics. Use incremental cursors (`lastupdated` on the World Bank endpoints) to pull only changed series. Airflow **dynamic task mapping** fans ingestion across many country/indicator combinations.

2. CDC & streaming. Scale Kafka horizontally (more partitions keyed by primary key preserves per-key ordering) and run multiple Kafka Connect workers for Debezium throughput. Add a **schema registry** (Avro) for safe schema evolution, and **dead-letter topics** for poison messages. Consumer-group lag metrics drive autoscaling of consumers.

3. ClickHouse storage & compute. Move from single-node to a **sharded + replicated** cluster: ReplicatedMergeTree tables coordinated by ClickHouse Keeper, fronted by Distributed tables to spread writes/reads. Adopt **tiered storage** (hot SSD → cold object storage/S3) with TTL ... TO VOLUME to keep hot data small while retaining history cheaply. Use **async inserts** or **Buffer** tables to absorb high-frequency small writes, and **projections/MVs** to pre-compute new access patterns. Read scaling comes from adding replicas.

4. Transformation & orchestration. dbt **incremental models** (with `insert_overwrite`/partition-aware strategies) process only new partitions; increase dbt threads for parallelism and use model selection to run subsets. Scale Airflow with the **Celery** or **Kubernetes** executor, deferrable operators, and pools to cap concurrency against shared resources.

Cross-cutting: separate compute for ingestion vs. query so heavy analytics never starves the write path; downsample/roll up aged data into coarser marts; enforce idempotent loads and a documented backfill procedure so volume spikes and replays are safe. Cost and performance are tuned by partition granularity, codec selection, TTL policy, and query concurrency limits — all adjustable without changing the pipeline's shape.